

자 기 소 개 서

최성광

sungkwang0908@naver.com | 경력 3년 10개월 (의료정보시스템 운영·장애대응·배포관리) | Microsoft AI School 10기
(2026.09 수료 예정)

github.com/CHOISUNGKWANG/portfolio-2026

“멈추면 안 되는 의료정보시스템을 약 4년간 운영해온 개발자입니다. 장애 대응과 배포 관리로 검증된 운영 역량을, 클라우드와 AI 영역까지 확장해가고 있습니다.”

1. 성장 과정 및 지원 동기 — 4년간 멈추면 안 되는 시스템을 지켜왔습니다

저는 **의료정보시스템을 약 4년간 운영해온 개발자**입니다. 서비스가 멈추면 진료가 멈추는 환경에서, 장애 대응과 배포 관리를 맡아 되돌릴 수 없는 사고 없이 시스템을 지켜왔습니다.

에스파이온코리아에서 CAMIS 의료정보시스템의 운영과 개발을 담당했습니다. 서버 로그와 배치 서비스를 상시 모니터링해 오류를 사전에 탐지하고, 장애가 발생하면 원인을 신속히 분석해 대응했습니다. 배포는 항상 롤백을 전제로 진행했고, 의료 개정 사양이 내려오면 비즈니스 로직을 수정해 기한 내 반영했습니다. 이 과정에서 몸에 붙은 원칙은 **‘내 변경이 지금 돌아가는 서비스에 어떤 영향을 주는가’를 먼저 확인**한다는 것입니다.

운영은 코드만 잘 써서 되는 일이 아니었습니다. 기관마다 업무 방식이 달라 같은 기능도 다르게 동작해야 했고, 그래서 저는 요구사항을 받기 전에 업무 흐름을 먼저 이해하는 방식으로 일했습니다. 일본 의료기관 5곳의 신규 오픈 온보딩을 담당하며 데이터를 세팅하고 실시간으로 이슈에 대응했고, 그중 한 곳은 현지에 직접 출장해 의료진과 얼굴을 맞대고 요구사항을 정리했습니다. **기술보다 업무 흐름이 먼저**라는 것을 현장에서 배웠습니다.

약 4년간 운영을 담당하며, 제 역량을 한 단계 넓힐 다음 축이 필요하다고 판단했습니다. 온프레미스 환경에서 쌓은 운영 감각을 클라우드 환경으로 옮기고, AI를 서비스에 안전하게 붙이는 방법까지 다룰 수 있어야 앞으로 더 넓은 범위를 책임질 수 있다고 봤습니다. 그래서 **Microsoft AI School 10기**에 지원했고, 2026년 3월 퇴사와 동시에 공백 없이 과정에 참여했습니다. **운영에서 쌓은 것을 버리는 선택이 아니라, 그 위에 더 얹기 위한 선택**이었습니다.

그 6개월 동안 저는 클라우드 위에 서버를 직접 세우고 AI 기능을 붙여 서비스를 완성해봤습니다. 이제 저는 **운영으로 검증된 신중함과 클라우드·AI로 확장한 구축 경험**을 함께 가진 개발자로서 다음 자리를 찾고 있습니다.

2. 직무 역량 — 서버 · API · 배포 · 장애 대응

저의 핵심 역량은 **운영 중인 서비스를 안정적으로 유지하고 개선하는 일** 전반입니다.

운영 및 장애 대응. 약 4년간 ASP.NET(C#), Oracle PL/SQL, DevExpress, Vue.js 기반 시스템을 담당했습니다. 서버 로그와 배치 서비스를 모니터링해 오류를 사전에 탐지하고, 장애 발생 시 로그와 쿼리 실행 계획을 추적해 원인을 좁혀 대응했습니다. 배포는 운영 안정성을 최우선으로 관리했으며, 되돌릴 수 없는 변경을 만들지 않는 것을 기본 원칙으로 삼았습니다.

데이터 계층 설계 및 성능 개선. 저장 프로시저·평션·뷰·트리거를 직접 작성해 비즈니스 로직을 구현했고, 여러 기관의 데이터를 통합 처리하기 위해 DB Link를 설계·적용했습니다. 데이터가 늘어날수록 느려지던 조회 화면은 원인을 **반복 조회(N+1) 구조**로 규명해 목록·상세 조회를 하나의 통합 프로시저로 묶어 요청 수를 N+1에서 1로 줄였습니다. 연도·일별 통계 화면을 개발해 기관 운영진의 의사결정 데이터도 제공했습니다.

서버 구축 및 API 설계. AI School 1차 프로젝트에서는 프론트엔드를 담당하면서도 서비스를 직접 End-to-End로 구축해보고 싶어 팀에 제안해 서버 구축까지 맡았습니다. **Azure VM 위에 Nginx 리버스 프록시와 FastAPI, PostgreSQL로 서버 환경을 직접 구성**하고, 도메인과 HTTPS를 연결해 실제 서비스를 공개했습니다. 2차 프로젝트에서는 백엔드를 전담해 **REST API를 설계하고 PostgreSQL 연동, 외부 API 오케스트레이션, 예외 처리와 장애 복구 로직을 구현**했습니다. 외부 API 쿼터가 소진되면 서비스가 멈추는 문제는 키 풀을 두고 쿼터 오류만 선별해 자동 전환하도록 처리했고, 수 GB급 모델은 요청마다 로드하지 않도록 앱 수명주기에 바인딩해 요청당 지연을 없앴습니다. HTTPS 프론트가 HTTP 백엔드를 호출할 수 없는 문제도 VM에 Caddy를 직접 설치해 해결했습니다.

AI 활용. AI는 제 역량의 확장 축입니다. 이미지 분류 모델을 서비스에 연동하고, GPU 환경에서 렌더링·추론 파이프라인이 안정적으로 서빙되도록 구성하는 역할을 맡았습니다. 모델 자체를 연구하기보다 **이미 있는 AI 기능을 서비스에 안전하게 붙이고 운영 가능한 형태로 만드는 쪽에** 강점이 있다고 생각합니다.

익숙한 언어와 프레임워크가 아니어도, 문제를 구조로 나눠 하나씩 검증하며 동작하는 결과물까지 만들어낼 수 있다는 것을 두 프로젝트로 확인했습니다.

3. 문제 해결 경험 — 되돌릴 수 있게 만들어두는 것부터 시작합니다

운영 환경에서 제가 가장 먼저 확보하는 것은 **'문제가 생겼을 때 되돌릴 수 있는가'**입니다.

CAMIS의 프로시저와 평션은 형상관리 대상이 아니어서, 수정한 뒤 문제가 생기면 이전 로직으로 돌아갈 근거가 남지 않았습니다. 저는 수정 전에 기존 객체를 백업본으로 먼저 생성하고 그 위에 변경분을 얹는 절차를 규칙으로 정해 지켰습니다. 그 결과 **배포 시점마다 복구 가능한 이전 버전이 항상 남아 있는 상태**를 유지할 수 있었습니다.

운영 DB 마이그레이션도 같은 원칙으로 접근했습니다. 실 데이터를 직접 변경하면 어떤 행이 이번 작업으로 바뀐 것인지 구분할 수 없기 때문에, 대상 데이터에 **작업일자**와 **작업자 이니셜**을 **식별값**으로 남겨 문제가 생기면 해당 배치만 정확히 선별해 롤백할 수 있게 했습니다. 데이터량이 많은 기관은 단일 스크립트로 일괄 처리하면 중간 실패 시 어디까지 반영됐는지 파악할 수 없어 재실행 자체가 위험했습니다. 그래서 전체를 **일정 건수 단위로 분할 실행**해 실패 지점을 특정하고 해당 구간만 다시 처리하도록 바꿨습니다. 도구가 없다면 절차로 만들어야 한다고 생각했고, 이 습관 덕분에 운영 중 되돌릴 수 없는 사고를 만들지 않았습니다.

성능 문제도 증상을 덮기보다 원인을 규명하는 쪽을 택했습니다. 특정 조회 화면이 데이터가 늘어날수록 느려졌는데, 원인은 목록을 조회한 뒤 각 항목마다 상세 정보를 다시 조회하는 **N+1 쿼리 구조**였습니다. 인덱스만 손대는 대신 여러 조회를 하나의 저장 프로시저로 통합하고 다건 결과를 커서로 반환하도록 바꿔, 요청 수를 데이터 규모와 무관하게 1회로 고정했습니다.

요구사항이 상충할 때는 커뮤니케이션으로 풀었습니다. 일본 기관 온보딩에서는 현지 담당자가 요청한 방식과 기존 시스템 구조가 맞지 않는 경우가 반복됐습니다. 한 번의 회의로 결론을 내려 하지 않고 **지속적으로 소통하며 요구사항의 의도를 확인하고 구현 가능한 형태로 함께 조정**해 나갔고, 결과적으로 기관 업무에 맞는 형태로 서비스를 안착시켰습니다.

AI 프로젝트에서도 같은 방식이 통했습니다. 학습 지표는 높는데 실제 촬영 사진에서 인식률이 떨어지는 문제가 있었는데, 파라미터를 조정하기 전에 원인을 추적해 학습 데이터의 배경이 지나치게 단조로워 생긴 과적합임을 규명하고, 실사용 환경 배경을 수집해 합성하는 방식으로 데이터를 보강해 해결했습니다. **증상이 아니라 원인을 먼저 찾는**다는 원칙은 운영과 프로젝트 모두에서 같았습니다.

4. 협업 경험 — 대립은 조율하고, 일정은 지키는 사람

저는 협업에서 두 가지를 중요하게 생각합니다. **기술적 의견 차이는 근거로 좁히고, 일정에 관한 것은 우선순위로 정리**하는 것입니다.

실무에서는 협업의 상대가 대부분 개발자가 아니었습니다. 의료진과 기관 담당자는 시스템 용어가 아니라 업무 언어로 요구사항을 전달했기 때문에, 요청을 그대로 받아 구현하기보다 **어떤 업무를 해결하려는 것인지 먼저 확인**하고 구현 방식을 제안했습니다. 신규 오픈 현장에서는 예상하지 못한 이슈가 즉시 발생하는데, 이때 개발자 판단만으로 처리하지 않고 담당자와 영향 범위를 함께 확인한 뒤 대응 순서를 정했습니다.

1차 프로젝트에서는 백엔드의 데이터 조회 방식을 두고 팀원과 의견이 갈렸습니다. 저는 SQL을 명시적으로 관리하는 방식을, 다른 팀원은 ORM을 주장했습니다. 저는 복잡한 조회 쿼리를 직접 제어하고 튜닝할 수 있어야 한다고 봤고, 팀원은 개발 속도와 코드 일관성을 중시했습니다. 어느 쪽도 틀린 주장이 아니었기에 장단점을 정리해

공유한 뒤, 단순 CRUD는 ORM으로 복잡한 조회는 SQL Mapper로 처리하는 구조를 함께 채택했습니다. 기술적 대립은 승패를 가리는 문제가 아니라 **적용 범위를 나누는 문제**일 수 있다는 것을 배웠습니다.

2차 프로젝트에서는 반대로 분명하게 선을 그어야 했습니다. 저는 핵심 파이프라인 개발과 프로젝트 관리를 맡고 있었고 일정이 빠듯한 상황에서 팀원이 새로운 기능을 제안했습니다. 아이디어는 좋았지만 착수하면 핵심 기능 조차 완성하지 못할 위험이 컸습니다. 남은 일정과 단계별 작업량을 함께 확인하며 '먼저 동작하는 프로토타입을 완성하고 시간이 남으면 붙이자'고 제안해 우선순위를 합의했고, 결과적으로 기한 내 완성한 뒤 확보된 시간에 확장 기능까지 추가할 수 있었습니다.

좋은 협업은 모든 의견을 받아들이는 것이 아니라, 무엇을 지금 하고 무엇을 나중에 할지 함께 합의하는 것이라고 생각합니다.

5. 입사 후 목표 — 팀이 믿고 맡길 수 있는 사람

저는 **서비스를 안정적으로 지키는 일에서 팀이 가장 먼저 믿고 맡기는 개발자**가 되고자 합니다.

약 4년간 운영 현장에 있으면서, 잘 만든 기능보다 잘 버티는 시스템이 더 큰 가치를 준다는 것을 배웠습니다. 장애가 났을 때 원인을 좁혀가는 순서, 배포 전에 되돌릴 방법을 먼저 확보하는 습관, 변경의 영향 범위를 미리 확인하는 태도 — 이런 감각은 기술 스택이 바뀌어도 그대로 유효하다고 생각합니다.

입사 후에는 먼저 담당 서비스의 도메인과 코드베이스, 배포 파이프라인을 빠르게 파악하는 데 집중하겠습니다. 신규 의료기관 온보딩마다 각 기관의 업무 흐름을 이해하는 것에서 출발했던 방식이 새로운 조직에 적응하는 데도 유효할 것이라 생각합니다.

그다음으로는 **장애 대응과 배포 관리에서 팀이 안심하고 맡길 수 있는 역할**을 맡고 싶습니다. 운영은 눈에 잘 드러나지 않지만 한 사람이 원칙을 지키는 것만으로 팀 전체의 리스크가 줄어드는 영역이라고 생각합니다. 여기서 신뢰를 쌓은 뒤, 축적한 운영 관점을 살려 기존 서비스를 개선하고 새로운 기술을 안전하게 도입하는 일까지 범위를 넓혀가겠습니다.

현재는 Microsoft AI School의 마지막 프로젝트를 진행 중이며 9월 초 수료를 앞두고 있습니다. 재직 중이 아니기에 입사 시점 조율도 자유롭습니다. 약 4년의 운영 경험으로 서비스를 지키고, 확장한 기술로 한 걸음 더 나아가는 데 기여하겠습니다.